Guía de Actividades Práctico-Experimentales Nro. 002

1. Datos Generales

Asignatura	Sistemas Distribuidos
Ciclo	6to
Unidad	Uno
Resultado de aprendizaje de la unidad	R2. Escribe un programa que realiza cualquier requerimiento de clasificación y conversión en unidades de mensajes, como paquetes, para comunicar información interesante entre dos hosts, bajo los principios de solidaridad, transparencia, responsabilidad y honestidad.
Práctica Nro.	2
Título de la Práctica	Arquitectura de Sistemas Distribuidos (comunicación cliente–servidor con archivos)
Nombre del Docente	José O. Guamán Q.
Fecha	06 de octubre de 2026 09 de octubre de 2026
Horario	Lunes 10h30 – 11h30 Viernes 07h30 – 09h30
Lugar	Aula
Tiempo planificado en el Sílabo	3 horas

2. Objetivo(s) de la Práctica:

Comprender la arquitectura de sistemas distribuidos mediante el desarrollo independiente de sus componentes (cliente y servidor).

3. Materiales y reactivos:

Guia practica

Se requiere un sistema que permita crear un servidor y un cliente con un canal de comunicación (archivo de texto), se debe evidenciar que se esta llenado el archivo.

4. Equipos y herramientas

Hardware

- Computador

Software

- Lenguaje de programación
- IDE de programación
- Archivo de texto

5. Procedimiento / Metodología

Parte 1: Desarrollo del SERVIDOR (1 hora 20 minutos) Este módulo se desarrolla de forma independiente Actividad

1. Crear un archivo:
   servidor.py
2. Crear un archivo auxiliar:
   entrada_servidor.txt
3. Implementar un programa que:
   - Revise periódicamente el archivo entrada_servidor.txt.
   - Detecte si existe contenido nuevo.
   - Procese el mensaje:
     - Ejemplo: convertir texto a mayúsculas o invertirlo.
   - Escriba el resultado

   en: salida_servidor.txt
4. Condiciones:
   - Debe ejecutarse continuamente (simulación de servicio activo).
   - No debe reprocesar el mismo mensaje.
   - Debe manejar archivos vacíos.

🔷 **Parte 2: Desarrollo del CLIENTE (1 hora)**

Se desarrolla sin necesidad de ver el código del servidor Actividad

1. Crear archivo:
   cliente.py
2. Implementar un programa que:
   - Solicite un mensaje al usuario.
   - Escriba el mensaje

   en: entrada_servidor.txt
   - Espere un tiempo (simulando latencia).
   - Lea la respuesta

   desde: salida_servidor.txt
   - Muestre el resultado en pantalla.
3. Condiciones:
   - No debe asumir cómo funciona el servidor internamente.
   - Solo debe conocer los archivos de entrada y salida.

🔷 **Parte 3: Integración del sistema (25 minutos)**

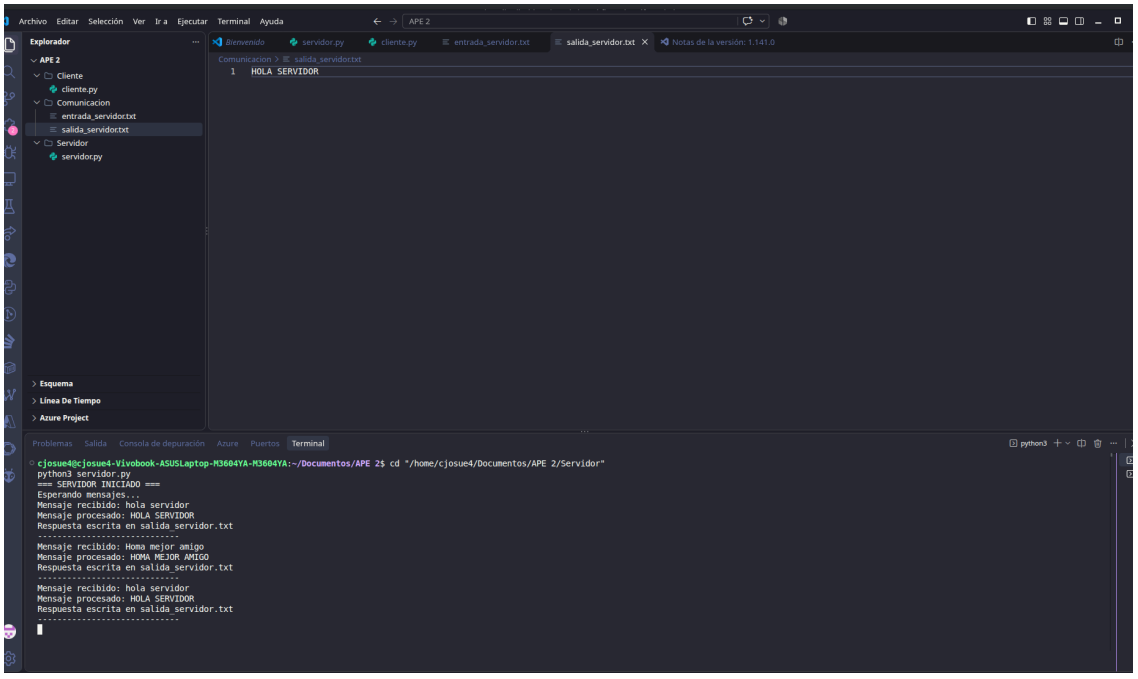
1. Ejecutar primero el servidor.
2. Ejecutar el cliente.
3. Enviar varios mensajes.
4. Verificar:
   - Flujo correcto
   - Respuesta adecuada

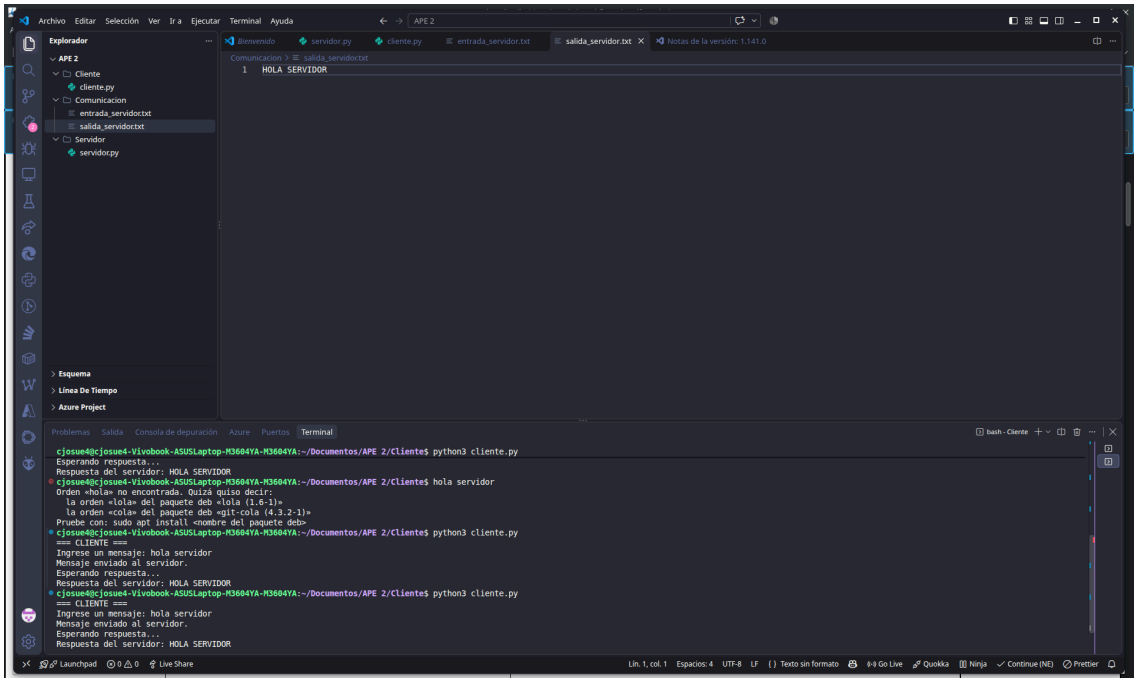
NOTA: los archivos de comunicación deben estar en una sola carpeta fuera de ambos proyectos

6. Resultados esperados:

Al finalizar la práctica, el estudiante entregará:

- Código fuente de servidor.py y cliente.py (repositorio Git). Link al repositorio.
- Capturas de pantalla que evidencien: (a) el archivo entrada_servidor.txt siendo llenado por el cliente, (b) salida_servidor.txt con el resultado procesado, (c) al menos 3 mensajes procesados correctamente.





- Respuestas a las preguntas de control en el reporte de práctica. Con ello demostrará: el desacoplamiento entre cliente y servidor, la comunicación indirecta por medio compartido, y las limitaciones del modelo (tiempo real, conflictos de escritura, dependencia del medio).

7. Preguntas de Control:

1. ¿Qué significa que cliente y servidor estén desacoplados?

Significa que el cliente y el servidor funcionan de manera independiente, ya que cada uno tiene su propio código y cumple una función diferente. En esta práctica, el cliente envía mensajes y el servidor los procesa mediante archivos de texto, sin necesidad de conocer cómo está programado internamente el otro componente.

2. ¿Qué ventajas tiene desarrollar componentes por separado?

Una ventaja es que permite desarrollar, probar y corregir cada componente de forma independiente. También facilita el mantenimiento del sistema, porque se pueden realizar cambios en el cliente sin modificar necesariamente el servidor, siempre que se mantenga la forma de comunicación entre ambos.

3. ¿Qué problemas pueden surgir al integrar sistemas independientes?

Pueden surgir problemas con las rutas de los archivos, el formato de los mensajes o el tiempo que tarda el servidor en responder. También pueden producirse conflictos si el cliente escribe un mensaje mientras el servidor está leyendo el archivo, lo que podría ocasionar respuestas incorrectas o pérdida de información.

4. ¿Qué sucede si el servidor no está ejecutándose?

Si el servidor no está ejecutándose, el cliente puede escribir el mensaje en entrada_servidor.txt, pero no será procesado. Por lo tanto, el archivo salida_servidor.txt no recibirá una respuesta nueva y el cliente podría mostrar una respuesta anterior o quedarse esperando, dependiendo de cómo esté programado.

5. ¿Cómo se simula la comunicación en esta práctica?

La comunicación se simula mediante dos archivos de texto compartidos. El cliente escribe el mensaje en entrada_servidor.txt, mientras que el servidor revisa su contenido, procesa el mensaje y escribe el resultado en salida_servidor.txt. Después, el cliente lee la respuesta y la muestra en pantalla.

6. ¿Qué limitaciones tiene este modelo frente a uno real?

Este modelo depende de los archivos de texto y de la frecuencia con la que el servidor los revisa, por lo que la respuesta no es inmediata. Además, pueden existir conflictos de escritura, problemas de sincronización y dificultades para manejar varios clientes al mismo tiempo. Tampoco ofrece, por sí solo, mecanismos completos de seguridad y control de errores.

7. ¿Cómo se podría mejorar la sincronización?

Se podría mejorar utilizando identificadores para cada mensaje, confirmaciones de recepción y mecanismos que permitan saber cuándo el servidor terminó de procesar una petición. También se pueden utilizar bloqueos de archivos para evitar que varios procesos escriban al mismo tiempo. Otra opción sería utilizar colas de mensajes o sockets para gestionar

la comunicación de una manera más eficiente.

8. ¿Qué pasaría si múltiples clientes usan el sistema?

Si varios clientes utilizan el sistema al mismo tiempo, podrían sobrescribir los mensajes de entrada_servidor.txt o leer una respuesta que corresponde a otro cliente. Esto ocurre porque todos comparten los mismos archivos. Para solucionarlo, se podrían utilizar identificadores únicos por petición, archivos separados para cada cliente o una cola de mensajes que organice las solicitudes.

8. Evaluación

NO APLICA

9. Bibliografía

Semana 2. Guaman, Jose. 2026

10. Elaboración y Aprobación

Elaborado por	José Guamán **Docente**	Validar únicamente en Firmado electrónicamente JOSE OSWALDO GUAMAN QUINCHE
Ejecutado Por	Rene Rivera **Técnico Docente**	Validar únicamente en Firmado electrónicamente RAMIRO RENE RIVERA GUAMAN
Aprobado por	Edison L Coronel Romero **Director de Carrera**	Firmado electrónicamente por: EDISON LEONARDO CORONEL ROMER Validar únicamente con FirmaEC